

Implementation of Grover's Algorithm for Hash-Prefix Search

Nitay Bahaliker

December 29, 2025

LinkedIn: www.linkedin.com/in/nitay-bahaliker-30a259316

GitHub: https://github.com/nitayB14/Grover_Sha

Abstract

Quantum computers offer computational advantages over classical algorithms, particularly in search problems where the size of the solution space grows exponentially. One such problem is finding an input to a SHA-type hash function whose output contains a prefix of zeros, a task that requires searching a large input space.

In this project, we construct a custom hash function inspired by SHA using basic logical building blocks such as XOR, AND, addition, and bit shifting, while adapting it to the reversibility requirements of quantum computation. The function is implemented as a quantum oracle with explicit management of ancilla qubits and computational resources, ensuring full reversibility while minimizing the overall qubit count. Grover's algorithm is then applied to search for inputs that produce a zero-prefix in the output.

The results demonstrate that effective quantum search can be achieved over a reversible hash function, yielding amplitude amplification consistent with the theoretical predictions of Grover's algorithm within a reduced search space.

1 Introduction

The purpose of Grover's algorithm is to reduce the size of a search space from N to $\sqrt{N}$ by amplifying the amplitudes of "good" states relative to all other states. The algorithm works by distinguishing marked states that satisfy a given condition from the remaining states, and by performing an effective search within the corresponding subspace of possible solutions.

SHA-type hash functions exhibit a unique property: although they are constructed from simple logical building blocks such as XOR, AND, addition, and bit shifting, they are designed such that a small change in the input produces a significant and unpredictable change in the output. As a result, there is no direct method to determine which input leads to a specific output. In this project, we construct a sample hash function based on similar building blocks in order to investigate whether such a function can be integrated into a quantum search framework.

It is important to emphasize that the output of the quantum algorithm is not the hash value itself, but rather a distribution over input states that appear with the highest probability after amplitude amplification. These states correspond to inputs that satisfy the desired prefix condition.

The structure of the work is inspired by the process of Bitcoin mining, in which there is fixed information (data) and variable information (nonce). The system operates by combining the data and nonce, applying a reversible hash function, marking states that lead to outputs with a zero-prefix, and finally applying the diffuser as part of Grover's algorithm.

The goal of the project is to demonstrate that quantum advantage in search problems does indeed exist, but that it comes with significant computational costs, such as increased circuit depth and the need for careful management of reversibility and computational resources. According to the principle of information preservation, every quantum computation must retain all intermediate information, and this project explicitly addresses these constraints in practice.

2 Preliminaries

Input / Output – We assume the existence of fixed information (data) and variable information (nonce), which are combined and used as input to the hash function.

Hash Function – The hash function is implemented as a reversible quantum circuit, allowing its integration as an oracle within the structure of Grover's algorithm and enabling phase marking.

Phase Flip – States whose hash output begins with a specified zero-prefix are marked using ancilla qubits, and a phase inversion is applied to the states that satisfy the condition.

Uncomputation (Reversibility) – After phase marking, the hash function is applied in reverse in order to clean the ancilla qubits and restore the quantum state to its original input.

Diffuser – The diffuser stage performs amplitude amplification of the marked states as part of Grover’s algorithm.

Working Assumption – In this work, we examine whether this combination enables a meaningful computational improvement in search over a large input space.

3 Problem Definition

Search space (64/256)

The problem is defined over a finite search space limited to $2^6 = 64$ and $2^8 = 256$ possible inputs, represented by a fixed number of qubits.

In this project the total input space is $2^{12} = 4096$ or $2^{16} = 65536$, where 6–8 qubits represent the nonce, which serves as the variable search input, and an additional 6–8 qubits represent the data, which remains fixed during the execution of the algorithm but can be modified according to the user’s choice.

The restricted search space is chosen in order to:

- Enable a full implementation of the oracle without encountering unrealistic circuit depth.
- Allow empirical evaluation of the distribution of algorithmic outcomes.
- Maintain consistency between the theoretical model of Grover’s algorithm and its practical implementation.

Thus, the problem serves as a reduced yet representative model of a general search problem.

Hash-Like Function

Instead of using a full SHA function, a custom hash-like function is defined, preserving the main characteristics of cryptographic hash functions:

- Construction from basic logical building blocks such as XOR, AND, addition, and bit shifting.
- Sensitivity to small changes in the input (avalanche-like effect).
- Absence of a direct method to infer the input from the output.

The function is constructed to be fully reversible, implementable as a quantum circuit, and suitable for integration as an oracle within Grover's algorithm.

The purpose of this function is not to replicate a full SHA implementation, but rather to preserve the computational nature of the problem while adapting it to the constraints of quantum computation.

Search Objective: Prefix / Condition

The objective of the search is not to find a specific hash value, but to identify input states whose output satisfies a simple Boolean condition, such as beginning with a predefined number of zeros (zero-prefix).

This condition allows for a clear definition of "good" states, can be checked locally on specific output bits, and naturally supports phase marking using ancilla qubits.

Why This Fits Grover's Algorithm

The defined problem satisfies the fundamental assumptions of Grover's algorithm:

- A Boolean checking function (oracle) exists.
- Solution states can be clearly defined.
- There is no efficient classical method to determine in advance which inputs satisfy the condition.
- Search is performed over an unstructured input space.

Classically, solving this problem requires $O(N)$ oracle evaluations on average. In contrast, Grover's algorithm reduces the number of required oracle calls to $O(\sqrt{N})$ while preserving the same search model.

Therefore, prefix search over a hash-like function constitutes a natural and suitable test case for implementing and evaluating Grover's algorithm within the framework of reversible quantum computation.

4 System Architecture

Registers

The quantum system is composed of several functional registers, each having a specific role in the implementation of the search algorithm.

Search registers (nonce)

This register represents the search space of the algorithm. It includes n qubits (in the basic implementation $n = 6$), and is initialized to a uniform superposition using

Hadamard gates. The states of this register correspond to the candidate inputs over which Grover's search is performed.

Data register

This register represents fixed information (data) used as an additional input to the hash-like function. Its size is equal to that of the nonce register in the current implementation, though it can be expanded. During the execution of the hash function, the register is modified; however, after the Uncomputation stage, it is restored to its original input state.

Work / Computation Registers

To support reversible computation (e.g. addition operations), additional registers are required to store intermediate information. These registers include carry and work qubits, which preserve computational data in accordance with the principle of information conservation in reversible quantum computation. These qubits are not measured and are used solely to enable the restoration of the original input state during Uncomputation.

Ancilla registers

Ancilla qubits are used to assist in computation and phase marking. A dedicated ancilla qubit is employed to mark states whose hash output satisfies the zero-prefix condition, enabling the phase flip operation in Grover's oracle. Ancilla qubits are reset after each iteration and can be reused.

Information flow

At the beginning of the algorithm, the data register is initialized to a fixed value, while the nonce register is prepared in a superposition state. The hash-like function is then applied to both registers, producing intermediate results on the work registers.

In the next stage, the prefix condition is checked on selected bits of the computed output, and phase marking is performed using a dedicated ancilla qubit. Subsequently, the hash function is applied in reverse (Uncomputation) to clean the work registers and restore the input registers to their original states.

Finally, the diffuser is applied exclusively to the nonce register, amplifying the amplitudes of states that satisfy the search condition.

Classical–Quantum Loop

The system operates within a hybrid classical–quantum loop. The classical component selects the data values, defines the prefix condition, determines the number of Grover iterations, and collects and analyzes the measurement results. The quantum component performs the search process according to these classical parameters.

5 Oracle Construction

The oracle is constructed as a closed logical process following the sequence:

Input->computation->phase mark->reversible computation->return to input

While a theoretical oracle appears simple in an abstract model, implementing a practical oracle for a hash-like function requires strict adherence to the principle of reversibility. A valid oracle must return the entire quantum system to its original input state.

Consequently, every intermediate computation, every applied phase flip, and every auxiliary qubit used during the process must be carefully accounted for and fully uncomputed. The construction of a reversible oracle within Grover's algorithm therefore relies on three essential components.

Reversible Hash Logic

The motivation for constructing a dedicated hash-like function, rather than directly implementing a classical function such as SHA-256, arises from quantum resource limitations. A fully reversible implementation of SHA-256 would require a number of qubits sufficient to preserve the entire computation history, which significantly exceeds the number of logical qubits available in current quantum hardware.

In general, every operation involved in a hash function—such as shifting, XOR, or addition—must preserve information in order to allow a return to the original input state at the end of the oracle. Each logical gate is therefore examined from the perspective of reversibility: whether the operation preserves information or discards it.

For example, a shift operation is inherently reversible, as qubits merely exchange positions. XOR operations can be implemented reversibly using CNOT gates, which preserve information and can be applied in reverse. In contrast, addition is more complex: adding two qubit registers requires expansion to a full-adder structure, including the preservation of information from previous computations (carry-in) and the storage of intermediate results (carry-out).

As a result, the structure of the hash function differs from its classical counterpart. Each addition operation must preserve auxiliary qubits and additional registers in order to retain information. The purpose of this stage is to demonstrate that classical hash building blocks can be adapted into a reduced, fully reversible hash-like function whose output can be used for a reversible Boolean check within the oracle.

Flip ancilla mechanism

This stage defines what constitutes a valid solution for the algorithm by specifying the condition to be checked on the hash output. In this project, the condition is defined as a zero-prefix in the hash output (e.g. 0000XX)

For instance, if the desired condition is that the hash output begins with four zeros, the oracle must mark states in which the first four output qubits equal zero. This is achieved

using a dedicated ancilla qubit that flips only when all relevant qubits satisfy the condition.

Since standard entangling operations (such as multi-controlled CNOT gates) act on the value 1, the relevant output qubits are first inverted using NOT gates so that a zero value maps to one. Entanglement is then applied, followed by a phase flip implemented via a Z gate on the ancilla qubit. Because Grover's diffuser operates on quantum phase rather than classical bit values, this phase inversion effectively marks the valid solution states.

To preserve reversibility, all marking operations are applied in reverse: the entangling gates and NOT gates are uncomputed, returning the system to its prior state. It is important to note that this is the only stage in which the hash output appears within the algorithm, and even then it is never measured, but rather serves as an internal component of the quantum phase-marking process.

Uncomputation strategy

The Uncomputation stage is essential for maintaining the correctness and physical validity of the algorithm. Every reversible computation generates intermediate information that must be removed in a physically consistent manner in order to return the system to its original input state.

Grover's algorithm conceptually separates the state space into two subspaces: valid (marked) states and invalid states. If intermediate computational data is not properly uncomputed, the diffuser cannot correctly amplify the amplitudes of the marked states, and quantum interference is disrupted. For this reason, after phase marking, the hash function is applied in reverse and all working registers are cleared.

Together, these three components define the oracle:

- The reversible hash logic determines what is checked.
- The flip-ancilla mechanism determines which states are marked.
- The Uncomputation strategy enables proper interaction with the diffuser and execution of Grover's algorithm.

Without any one of these components, the oracle is incomplete and cannot support a correct quantum implementation of the search process.

6 Diffuser Design

Participating Qubits

The diffuser operates exclusively on the search registers (search / nonce registers), which represent the input space over which the search is performed. The purpose of the diffuser is to amplify the amplitudes of input states marked by the oracle; therefore, its operation is defined as a geometric transformation acting solely on the search space. The probability amplification is performed relative to the average amplitude of these states.

Non-Participating Qubits

The data registers, computation registers, and ancilla qubits do not participate in the diffuser, as they do not represent search variables but rather fixed information or intermediate computational data. After the Uncomputation stage, these registers are returned to known and uniform states, and including them in the diffuser would violate the fundamental assumptions of Grover's algorithm and disrupt the amplitude amplification process.

7 Full Grover Iteration

In this section, we present a full iteration of Grover's algorithm—from the preparation of the superposition state, through marking the solution using the oracle, to amplitude amplification via the diffuser. The purpose of this section is to illustrate how a fixed and repeated sequence of quantum operations concentrates the probability distribution toward states that satisfy the search condition, in a non-classical and efficient manner.

Order of operations

Each Grover iteration consists of an oracle application, which includes computation, phase marking, and Uncomputation, followed by the application of the diffuser, which amplifies the amplitudes of the marked states within the search space.

No measurement is performed during the execution of the algorithm, and the structure of operations remains unchanged between successive iterations. Each iteration acts on the same search space and applies a consistent transformation to the system's wave function.

Number of iterations

The number of Grover iterations is determined by the size of the search space N . For a single solution, the optimal number of iterations is approximately $\sqrt{N}$, allowing maximal concentration of amplitude on the valid states. However, the algorithm does not return a deterministic outcome, but rather a probability distribution resulting from the wave-like nature of the quantum state.

It is therefore important to distinguish between the number of quantum iterations within a single algorithm run and the number of shots (measurements) required to obtain a statistically significant result distribution. For example, in a search space of size 64, a

preference for the marked states can be achieved after approximately 8 Grover iterations, but in order to clearly observe the resulting distribution, a much larger number of measurements—such as 1024 shots—is required.

In significantly larger search spaces, this distinction becomes critical. Even though the optimal number of Grover iterations scales as $\sqrt{N}$, the number of measurements required to reliably identify the correct solution may grow substantially. Consequently, a trade-off exists between the minimal number of quantum operations needed to amplify the solution amplitude and the number of repetitions required to statistically distinguish the marked states from the remaining states.

8 Results & Validation

In the result analysis stage, it is necessary to precisely define which qubits are measured, since the circuit includes a large number of registers and ancilla qubits. The purpose of the algorithm is to identify valid inputs in the search space; therefore, measurements are performed exclusively on the qubits representing the nonce register, while the data register, computation registers, and ancilla qubits are returned to known states through Uncomputation.

Although the data register participates in the computation, the output of the algorithm is not the hash value itself but the input values in the nonce space whose measurement probabilities are increased through phase marking and the diffuser. Accordingly, the analysis focuses solely on the measurement distribution of the nonce register.

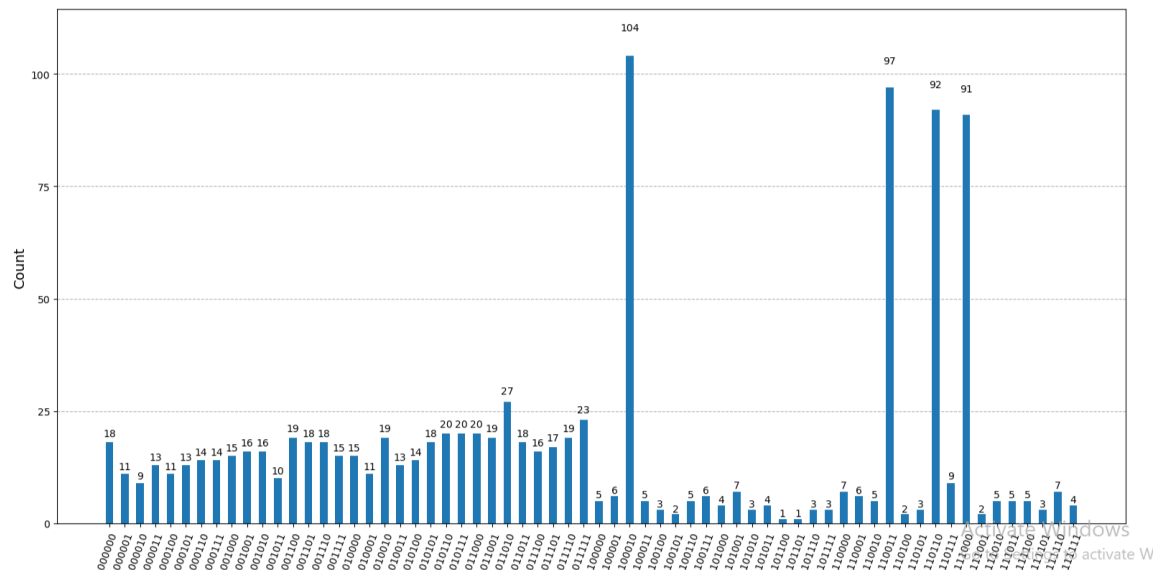


Figure x

Figure X presents a histogram of measurement counts for a search space of size 64. The results clearly show that a small subset of nonce values appears with significantly higher

frequency than the remaining states, indicating successful amplitude amplification of states that satisfy the predefined prefix condition.

Quantitatively, the marked states constitute a substantially higher fraction of the measurement outcomes compared to the uniform distribution. This behavior matches the expected performance of Grover's algorithm and is consistent with the theoretical reduction in oracle evaluations from $O(N)$ to $O(\sqrt{N})$ within the reduced search space framework.

To validate correctness, the most dominant nonce values identified by the quantum algorithm were evaluated using a classical implementation of the same hash-like function. This verification confirmed that the corresponding hash outputs satisfy the predefined prefix condition, demonstrating that the quantum algorithm successfully identifies valid input states.

These results confirm that the oracle construction and Grover iterations shape the system's wave function such that the probability of measuring valid states increases consistently, and that the observed measurement distributions faithfully reflect the defined search objective.

9 Common Pitfalls and Debugging Insights

Unitary oracle

Throughout this project, it became clear that preserving circuit unitarity is a necessary condition for a correct oracle implementation. Errors such as omitting the Uncomputation stage or attempting to reset ancilla qubits via measurement break the unitarity of the system and disrupt the behavior of the algorithm.

Therefore, the oracle must be constructed as a closed sequence of operations:

Function -> phase flip -> Uncomputation.

Ensuring that all computation qubits return to their original states before the diffuser is applied.

Qubits placement

During circuit construction, careful tracking of the role and position of each qubit is essential. For example, a qubit representing part of the nonce register must participate only in the intended computation steps and not be unintentionally reused due to incorrect register mapping. Inconsistent qubit placement can lead to incorrect oracle behavior and faulty phase marking.

Scaling and resource limits

This work highlights that even reduced models require a significant number of qubits. An implementation with two inputs (data and nonce) of 6 qubits each, including a single adder operation and ancilla qubits, requires approximately 20 qubits. Increasing the input size to 8 qubits per register raises this requirement to 26 qubits.

Since classical simulation of quantum circuits requires memory that scales exponentially with the number of qubits, it is crucial to match the model size to available hardware resources. For example, a system with 8 GB of RAM can typically simulate up to approximately 32 qubits.

Phase error

Incorrect phase marking is a common source of failure in Grover’s algorithm. Phase flips must be applied to the correct qubits after the condition-defining qubits are properly identified. Ancilla qubits serve as indicators of whether the condition is satisfied, and incorrect usage prevents the diffuser from amplifying the amplitudes of the desired states.

Additionally, during the diffuser operation, phase marking is applied across the nonce register: all qubits except the final one act as control conditions, while the final qubit is the target of the phase rotation.

10 Cross-Disciplinary Insights and Research Directions

Insight I – Simplifying Grover Program Construction

Constructing an oracle for Grover’s algorithm requires manual management of reversibility, ancilla qubits, and Uncomputation stages, making implementations highly error-prone and tightly coupled to low-level circuit design. This complexity highlights the need for abstraction in quantum program development.

One possible direction is a high-level programming model in which the user specifies the search space, the computation function, and the validity condition, without explicitly constructing the circuit. A translation layer could automatically generate a valid oracle, handle phase marking, manage computation qubits, perform Uncomputation, and apply the diffuser.

Such a layer, potentially supported by language models or similar semantic tools, could serve as an interface between problem specification and reversible quantum implementation. This approach shifts the focus from circuit construction to problem definition, while preserving fundamental physical constraints such as unitarity and information conservation.

Insight II – Reducing qubit resources through efficient operations

One of the main limitations encountered throughout the project was the rapid increase in the number of qubits required to preserve reversibility, particularly when implementing arithmetic operations such as addition. The need to store intermediate information (carry-in and carry-out) and perform full Uncomputation causes circuit sizes to grow significantly beyond the original input size.

A possible research direction is the development of qubit-efficient reversible operations that enable complex computations while minimizing auxiliary register usage. Examples include reuse of ancilla qubits, adders with reduced footprint, or structured computation strategies that limit the need to retain all intermediate states simultaneously.

Improving qubit efficiency is critical not only for classical simulation feasibility, but also for future execution on real quantum hardware, where qubit counts are limited and each additional qubit introduces physical cost and increased error rates.

11 Discussion

Limitations

In practice, modern computational systems operate under limitations in memory, computational resources, and scalability. These limitations become especially pronounced when applying reversible quantum algorithms, and can be divided into several main categories.

Reversible cost – When constructing Grover’s algorithm, even simple logical operations such as ADD or AND become expensive. The requirement to preserve information for reversibility leads to a significant increase in the number of qubits, circuit depth, and overall computational resource consumption.

Oracle-specific design – The presented oracle is not general-purpose, but rather specifically tailored to a hash-like function with a prefix-based condition. Any change in the search condition or target function requires a complete redesign of the oracle, which limits the flexibility of the approach.

Shot complexity – Even though the number of required Grover iterations scales as $\sqrt{N}$, a substantially larger number of shots is needed to obtain reliable statistics. This requirement increases the total computation time, especially in the presence of hardware noise and errors.

Why this is not scale-ready

During the development of the algorithm, it became clear that the input space cannot be expanded directly without modifying both the oracle architecture and the overall circuit structure.

Each additional qubit added to the nonce or data register results in an exponential increase in the total number of qubits required for computation. While adding an adder increases the qubit count linearly, it contributes significantly to the growth of circuit depth.

The resulting increase in circuit depth occurs faster than the coherence time of realistic quantum hardware, making such implementations impractical at large scales. Furthermore, the requirement for reversibility demands a large number of ancilla qubits, which is not proportional to the problem size or to their effective computational contribution.

What Real Hardware Can Accomplish

Dedicated quantum hardware could, in principle, improve execution efficiency compared to software-based simulations.

For example, realistic hardware can execute gates on different qubits in parallel, such as applying multiple CX or NOT gates simultaneously rather than sequentially. In addition, advanced hardware architectures may support genuine multi-qubit gates, reducing the need for long gate sequences and extensive use of ancilla qubits.

Beyond these improvements, real quantum hardware could explore significantly larger search spaces than those currently feasible, thereby enabling new classes of algorithms that are directly tailored to the structure and capabilities of the underlying hardware.

12 Conclusion

- In this project, we implement a full quantum search based on Grover's algorithm, including a reversible oracle for a hash-like function with a prefix condition, realized as an end-to-end system rather than merely a theoretical model.
- Under the stated assumptions, the results confirm the expected amplification behavior of order $\sqrt{N}$, demonstrating the correctness of both the phase-marking mechanism and the oracle design.
- This implementation highlights the cost of reversibility: even simple logical operations lead to an increase in the number of qubits, circuit depth, and overall resource consumption.
- Scaling analysis shows that the method is not scale-ready on contemporary quantum hardware due to coherence-time limitations, extensive use of ancilla qubits, and high statistical requirements (shots).
- Despite these limitations, the project presents a clear methodology for constructing problem-specific oracles and provides a practical foundation for further research in hardware-aware directions.
- In conclusion, the project bridges the gap between the theoretical formulation of Grover's algorithm and realistic reversible implementations, clarifying both the potential and the current limitations of quantum search algorithms.

Acknowledgements

The author would like to thank Nisim Mizrahi Edri Ben-Zakai for discussions that, while not directly related to this work, somehow contributed to its completion.

References

- 1) L. K. Grover,
A fast quantum mechanical algorithm for database search,
Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC),
1996.
- 2) M. A. Nielsen and I. L. Chuang,
Quantum Computation and Quantum Information,
Cambridge University Press, 2010.
- 3) C. H. Bennett,
Logical reversibility of computation,
IBM Journal of Research and Development, 17(6), 1973.
- 4) G. Brassard, P. Høyer, M. Mosca, and A. Tapp,
Quantum Amplitude Amplification and Estimation,
Contemporary Mathematics, vol. 305, 2002.
- 5) A. Barenco et al.,
Elementary gates for quantum computation,
Physical Review A, 52(5), 1995.
- 6) T. Toffoli,
Reversible computing,
MIT Laboratory for Computer Science Technical Report, 1980.
- 7) A. G. Fowler et al.,
Surface codes: Towards practical large-scale quantum computation,
Physical Review A, 86, 2012.
- 8) S. Aaronson,
Lower bounds for local search by quantum arguments,
SIAM Journal on Computing, 35(4), 2006.
- 9) A. Montanaro,
Quantum algorithms: an overview,
npj Quantum Information, 2, 2016.
- 10) J. Preskill,
Quantum Computing in the NISQ era and beyond,
Quantum, 2, 79, 2018.